Bubble Sort in C

28	6	4	2	24

Pass 1

28 & 2 swapped

28	6	4	2	24

28 & 4 swapped

6	28	4	2	24

28 & 2 swapped

6	4	28	2	24

28 & 24 swapped

6	4	2	28	24

6	4	2	24	28

Pass 2

6 & 4 swapped

6	4	2	24	28

6 & 2 swapped

4	6	2	24	28

no Swapping

4	2	6	24	28

4	2	6	24	28

Pass 3

4 & 2 swapped

4	2	6	24	28

no swapping

2	4	6	24	28

2	4	6	24	28

Pass 4

no swapping

2	4	6	24	28

2	4	6	24	28

Final Result

2	4	6	24	28

PrepInsta

Press Esc to exit full screen

Bubble Sort in C

			0	1	2	3	4	5	6	7		
			10	6	2	18	16	4	8	14		
Pass 1												
i = 0	(Swap)	j = 0	6	10	2	18	16	4	8	14	Iteration 1	18
	(Swap)	j = 1	6	2	10	18	16	4	8	14	Iteration 2	
	(Swap)	j = 2	6	2	18	10	16	4	8	14	Iteration 3	
	(Swap)	j = 3	6	2	10	16	18	4	8	14	Iteration 4	
	(Swap)	j = 4	6	2	10	16	4	18	8	14	Iteration 5	
	(Swap)	j = 5	6	2	10	16	4	8	18	14	Iteration 6	
	(Swap)	j = 6	6	2	10	16	4	8	14	18	Iteration 7	
Pass 2	(Swap)	j = 0	2	6	10	16	4	8	14		Iteration 1	16
i = 1	(No Swap)	j = 1	2	6	10	16	4	8	14		Iteration 2	
	(No Swap)	j = 2	2	6	10	16	4	8	14		Iteration 3	
	(Swap)	j = 3	2	6	10	4	16	8	14		Iteration 4	
	(Swap)	j = 4	2	6	10	4	8	16	14		Iteration 5	
	(Swap)	j = 5	2	6	10	4	8	14	16		Iteration 6	
Pass 3	(No Swap)	j = 0	2	6	10	4	8	14			Iteration 1	14
i = 2	(No Swap)	j = 1	2	6	10	4	8	14			Iteration 2	
	(Swap)	j = 2	2	6	4	10	8	14			Iteration 3	
	(Swap)	j = 3	2	6	4	8	10	14			Iteration 4	
	(No Swap)	j = 4	2	6	4	8	10	14			Iteration 5	

Pass 3 i = 2	(No Swap)	j = 0	2	6	10	4	8	14	Iteration 1	14
	(No Swap)	j = 1	2	6	10	4	8	14	Iteration 2	
	(Swap)	j = 2	2	6	4	10	8	14	Iteration 3	
	(Swap)	j = 3	2	6	4	8	10	14	Iteration 4	
	(No Swap)	j = 4	2	6	4	8	10	14	Iteration 5	
Pass 4 i = 3	(No Swap)	j = 0	2	6	4	8	10		Iteration 1	10
	(Swap)	j = 1	2	4	6	8	10		Iteration 2	
	(No Swap)	j = 2	2	4	6	8	10		Iteration 3	
	(No Swap)	j = 3	2	4	6	8	10		Iteration 4	
Pass 5 i = 4	(No Swap)	j = 0	2	4	6	8			Iteration 1	8
	(No Swap)	j = 1	2	4	6	8			Iteration 2	
	(No Swap)	j = 2	2	4	6	8			Iteration 3	
Pass 6 i = 5	(No Swap)	j = 0	2	4	6				Iteration 1	6
		j = 1	2	4	6				Iteration 2	
Pass 7 i = 6	(No Swap)	j = 0	2	4					Iteration 1	4

At the end of each pass largest element reaches end of the array

Swapping only happens if left element > right element

Insertion Sort in C

- Initial Array

8	6	4	20	24	2	10	12

- Since, 6 < 8

8	6	4	20	24	2	10	12

6 will get inserted before 8

- Since, 4 < 6

6	8	4	20	24	2	10	12

4 will get inserted before 6

- 20 is at correct position, no insertion needed

4	6	8	20	24	2	10	12

- 24 is at correct position, no insertion needed

4	6	8	20	24	2	10	12

- Since, 2 < 4

4	6	8	20	24	2	10	12

2 will get inserted before 4

- Since, 10 < 20

2	4	6	8	20	24	10	12

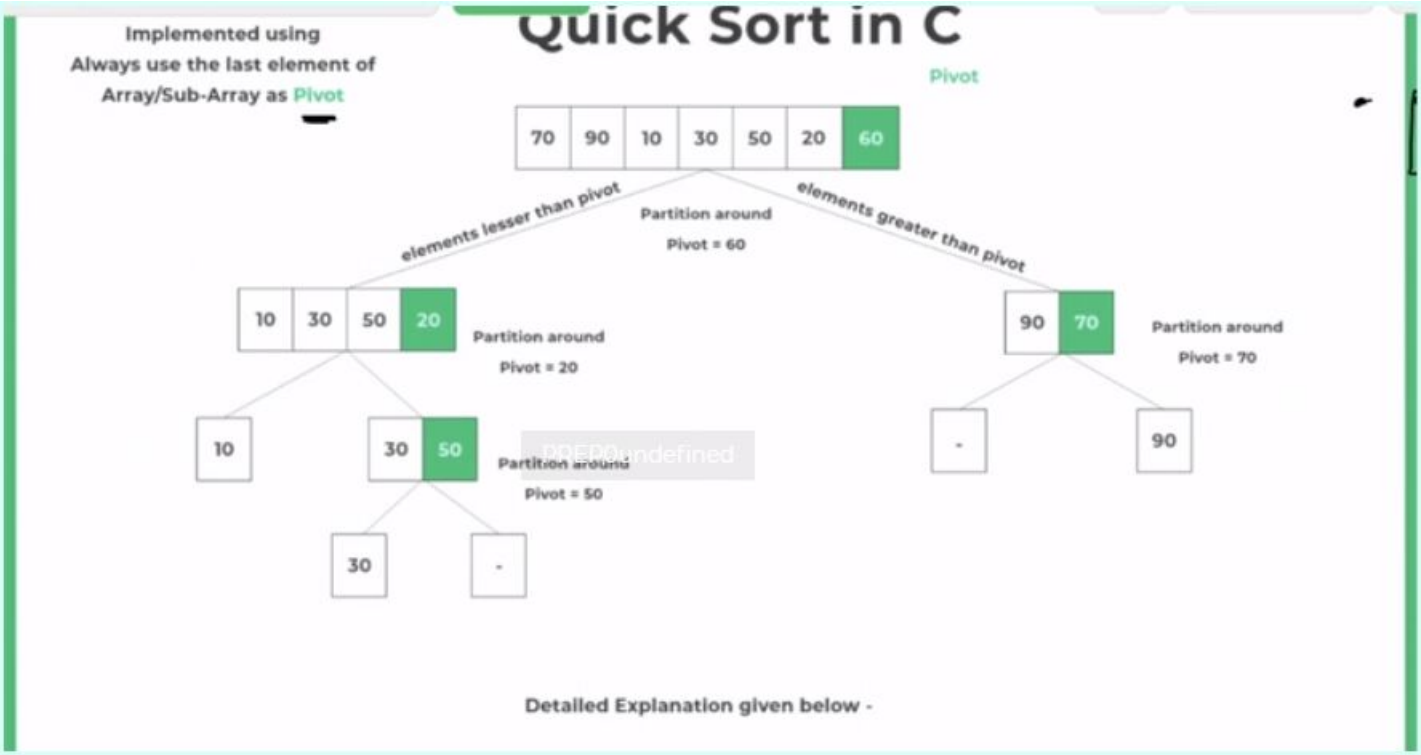
10 will get inserted before 20

- Since, 12 < 20

2	4	6	8	10	20	24	12

12 will get inserted before 20

2	4	6	8	10	12	20	24


Quick Sort in C
Implemented using
Always use the last element of
Array/Sub-Array as Pivot
Pivot
70 90 10 30 50 20 60
elements lesser than pivot
Partition around
Pivot = 60
elements greater than pivot
10 30 50 20
Partition around
Pivot = 20
90 70
Partition around
Pivot = 70
10
30 50
Partition around
Pivot = 50
-
90
30
-
Detailed Explanation given below -

Quick Sort in C

swapIndex = low - 1

	Action		swapIndex
		70 90 10 30 50 20 60	
No	None	70 90 10 30 50 20 60	swapIndex = -1
No	None	70 90 10 30 50 20 60	swapIndex = -1
Yes	swapIndex++ Swap 10 & 70	70 90 10 30 50 20 60	swapIndex = 0
Yes	swapIndex++ Swap 30 & 90	10 90 70 30 50 20 60	swapIndex = 1
Yes	swapIndex++ Swap 50 & 70	10 30 70 90 50 20 60	swapIndex = 2
Yes	swapIndex++ Swap 20 & 90	10 30 50 90 70 20 60	swapIndex = 3
	swap arr[swapIndex + 1] & pivot	10 30 50 20 70 90 60	
		10 30 50 20 60 90 70	

Left Partition: 10 30 50 20

Right Partition: 90 70

Prepinsta

Export board

Invite

Quick Sort in C

Steps not completely explained
for students own good imagination

swap arr[swapindex + 1] & pivot

swap arr[swapindex + 1] & pivot

Finally, doing the same will result in

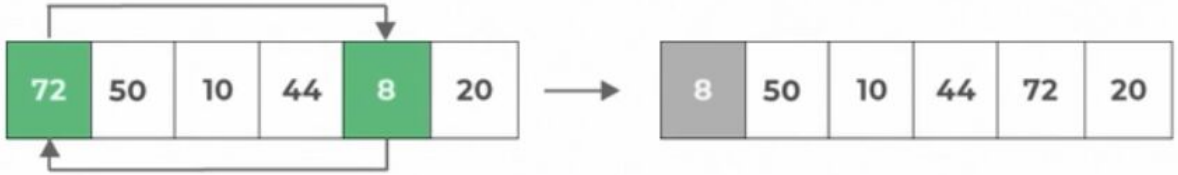
Re-writing the array after partitioning and swapping again -

Selection Sort in C

72	50	10	44	8	20

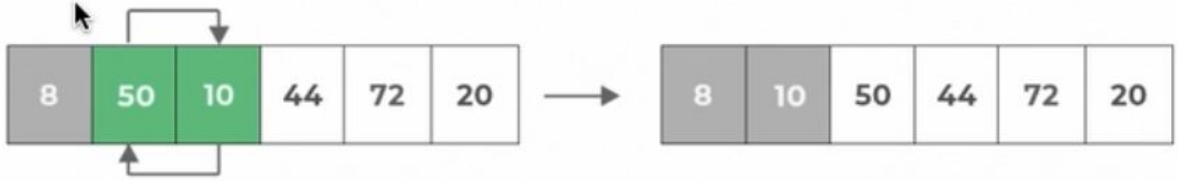
8 is min. was swapped with element at 0th index

- Sorted elements [] = 8
- UnSorted elements [] = 50 10 44 72 20



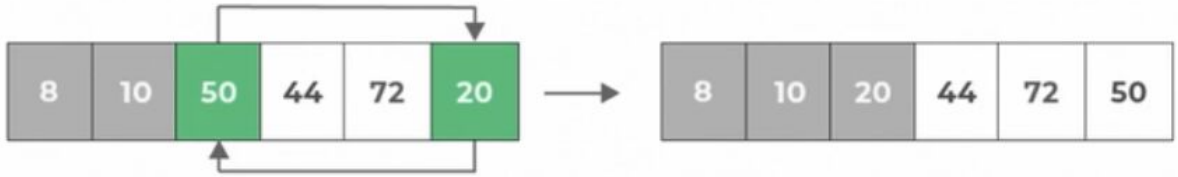
10 is min. was swapped with element at 1st index

- Sorted elements [] = 8 10
- UnSorted elements [] = 50 44 72 20



20 is min. was swapped with element at 2nd index

- Sorted elements [] = 8 10 20
- UnSorted elements [] = 44 72 50

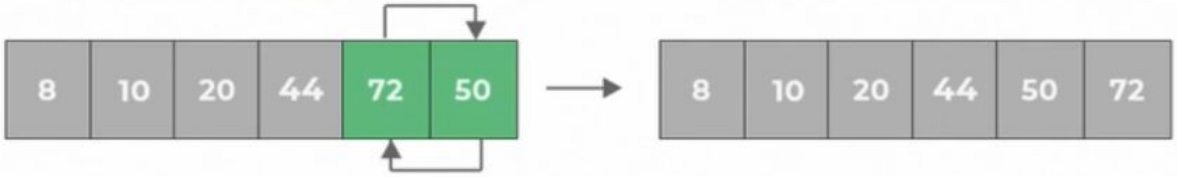


44 is min. exists at 3rd index no need for swapping

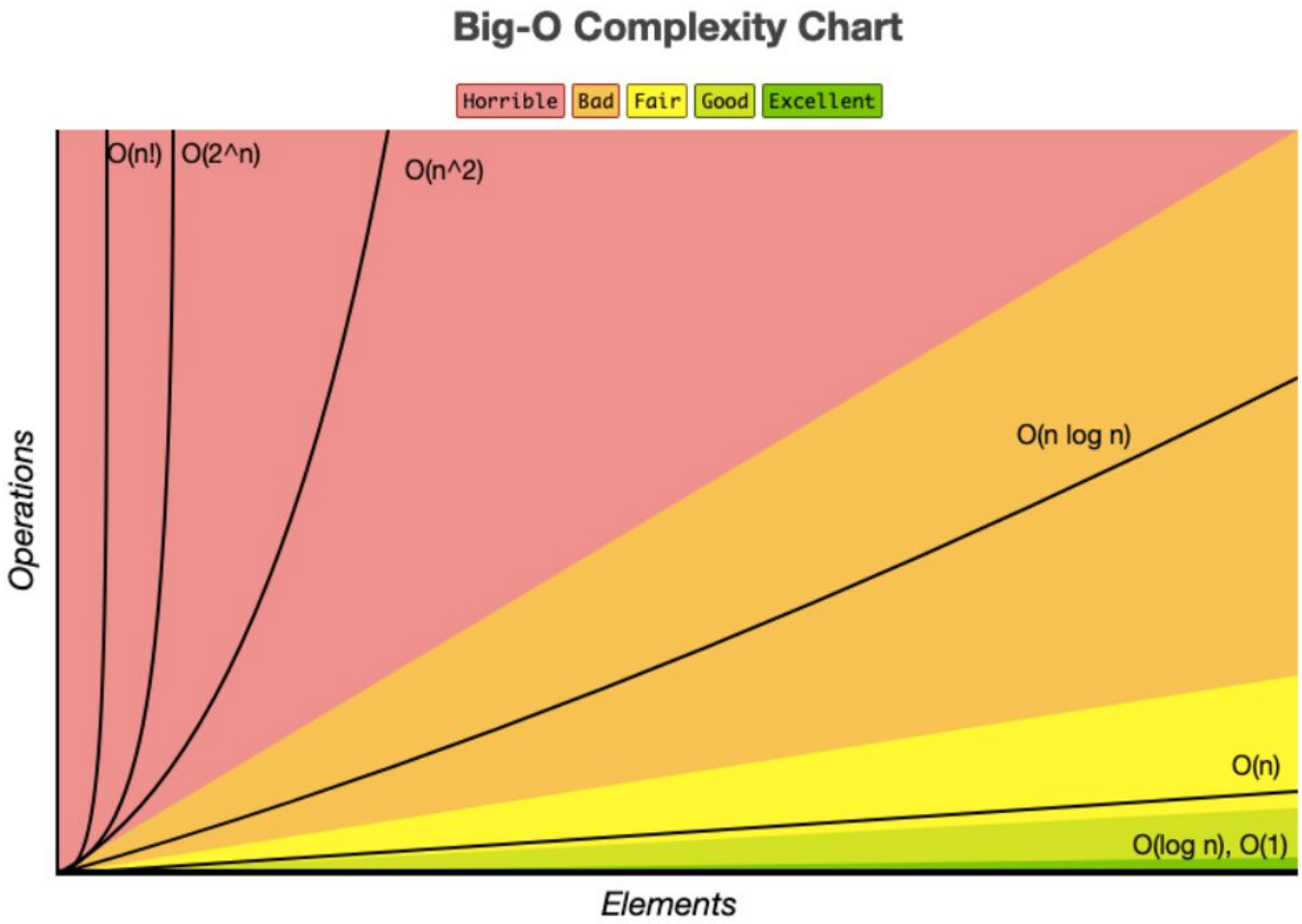
- Sorted elements [] = 8 10 20 44
- UnSorted elements [] = 72 50

50 is min. was swapped at with element at 4th index

- Sorted elements [] = 8 10 20 44 50 72
- UnSorted elements [] =



Final Result


Big-O Complexity Chart
Horrible
Bad
Fair
Good
Excellent
O(n!)
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n), O(1)
Operations
Elements

Array Sorting Algorithms

Algorithm	Time Complexity			Space Complexity
	Best	Average	Worst	Worst
Quicksort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(log(n))
Mergesort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(n)
Timsort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)
Heapsort	Ω(n log(n))	Θ(n log(n))	O(n log(n))	O(1)
Bubble Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Insertion Sort	Ω(n)	Θ(n^2)	O(n^2)	O(1)
Selection Sort	Ω(n^2)	Θ(n^2)	O(n^2)	O(1)
Tree Sort	Ω(n log(n))	Θ(n log(n))	O(n^2)	O(n)
Shell Sort	Ω(n log(n))	Θ(n(log(n))^2)	O(n(log(n))^2)	O(1)
Bucket Sort	Ω(n+k)	Θ(n+k)	O(n^2)	O(n)
Radix Sort	Ω(nk)	Θ(nk)	O(nk)	O(n+k)
Counting Sort	Ω(n+k)	Θ(n+k)	O(n+k)	O(k)
Cubesort	Ω(n)	Θ(n log(n))	O(n log(n))	O(n)

Common Data Structure Operations

Data Structure	Time Complexity								Space Complexity
	Average				Worst				Worst
	Access	Search	Insertion	Deletion	Access	Search	Insertion	Deletion	
Array	Θ(1)	Θ(n)	Θ(n)	Θ(n)	O(1)	O(n)	O(n)	O(n)	O(n)
Stack	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Queue	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Singly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Doubly-Linked List	Θ(n)	Θ(n)	Θ(1)	Θ(1)	O(n)	O(n)	O(1)	O(1)	O(n)
Skip List	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n log(n))
Hash Table	N/A	Θ(1)	Θ(1)	Θ(1)	N/A	O(n)	O(n)	O(n)	O(n)
Binary Search Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)
Cartesian Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(n)	O(n)	O(n)	O(n)
B-Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Red-Black Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
Splay Tree	N/A	Θ(log(n))	Θ(log(n))	Θ(log(n))	N/A	O(log(n))	O(log(n))	O(log(n))	O(n)
AVL Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(log(n))	O(log(n))	O(log(n))	O(log(n))	O(n)
KD Tree	Θ(log(n))	Θ(log(n))	Θ(log(n))	Θ(log(n))	O(n)	O(n)	O(n)	O(n)	O(n)